

HR Management System

Project Overview & Setup Documentation

Author: Hassan Raheem

Email: hmarwat49@gmail.com

Year: 2026

Version: 1.0.0

What is this Application?

The **HR Management System** is a full-stack web application built for Stella Technology as a final internship project. It digitises the core HR workflows of a company — employee onboarding, department management, leave tracking, and payroll generation — across three distinct user roles.

Admin

Manages the entire organisation. Registers employees, creates departments (each must have an HR assigned), and monitors all leaves and payroll in read-only view.

HR

Manages day-to-day operations for their department only. Approves or rejects leave requests and generates monthly payroll for their employees.

Employee

Personal self-service dashboard. Applies for leave, tracks request status, checks remaining leave balance, and views payroll history.

Key workflow: Admin registers HR employees first → Creates departments (each assigned one HR) → Registers regular employees → HR logs in and manages their department's leaves & payroll → Employees apply for leave from their own dashboard.

Tech Stack

Layer	Technology
Frontend	Angular 22 — standalone components, signals API, reactive forms

Layer	Technology
Backend	Spring Boot 4.1.1 — REST API, Spring Security 7, @PreAuthorize
Database	MongoDB 6+ — 7 collections, Spring Data MongoDB
Authentication	JWT (jjwt 0.12.6) — stateless Bearer token authentication
Build	Maven (backend) · Angular CLI / Vite (frontend)

Features

Module 1 — Authentication

- JWT-based stateless authentication with 24-hour token expiry
- Three roles: `ADMIN` , `HR` , `EMPLOYEE`
- Case-insensitive email login (e.g. `ADMIN@HRMS.COM` = `admin@hrms.com`)
- Route guards on the frontend — `authGuard` , `adminGuard` , `hrGuard`
- Default admin account auto-seeded on first startup

Module 2 — Employee Management (Admin)

- Register employee in one flow: creates login account + employee profile
- Employee types: `EMPLOYEE` , `MANAGER` , `HR`
- HR employees can be registered without a department — placed in Unassigned (DEPT-0000) until a department is created for them
- Update details, change type (triggers automatic history record), delete, add performance notes
- Search by name, filter pills: All / Employee / Manager / HR
- Unassigned employees banner alerts admin when staff have no department

Module 3 — Department Management (Admin)

- Create / update / delete departments
- **Every department requires a mandatory HR assignment** — cannot be saved without one
- Validation prevents the same HR from being assigned to more than one department
- Assign a manager from MANAGER-type employees (optional)
- Departments table shows Manager name and HR name columns

- Delete with employee reassignment — move staff to another department or the Unassigned bucket before deletion

Module 4 — Leave Management

- **Admin:** View all leaves (read-only), filter by status and department, leave cards show dept name and HR name, balance lookup for all employees
- **HR:** Approve / reject pending leaves for their department only (auto-scoped by backend), balance lookup for their department
- **Employee:** Apply for leave (Annual / Casual / Sick / Unpaid), view own history and remaining balance
- Leave balance auto-decremented when a request is approved
- Only PENDING requests can be actioned

Module 5 — Payroll Management

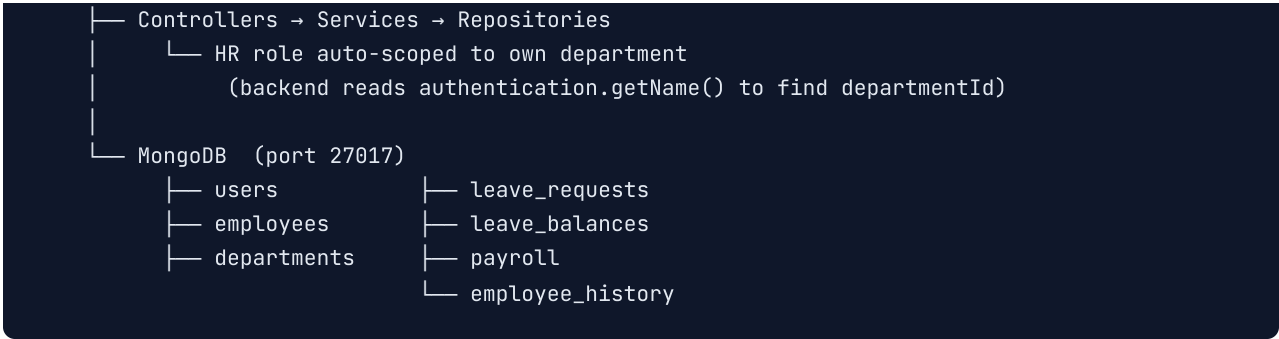
- **HR:** Generate payroll for department employees for a chosen month/year — employee dropdown shows only department members; mark as PAID / UNPAID
- **Admin:** View all payroll records (read-only), filter by month, year, status, and department
- **Employee:** View own payroll history with net salary breakdown
- Net salary = `Basic Salary + Allowances - Deductions`

Module 6 — Employee History & Records

- Automatic promotion record when employee type changes
- Automatic designation change record when designation is updated
- Admin can add free-text performance notes to any employee
- Full history timeline viewable per employee, filterable by record type

System Architecture

```
Browser (Angular 22)
  |
  | HTTP + JWT Bearer token
  ▼
Spring Boot REST API (port 8080)
  |
  ├── Spring Security 7
  |   └── JwtAuthFilter validates every request
  |
  |
```



JWT Authentication Flow

1. User logs in → backend returns a signed JWT token (24-hour expiry)
2. Angular stores the token in `localStorage`
3. A global HTTP interceptor automatically attaches `Authorization: Bearer <token>` to every request
4. `JwtAuthFilter` validates the token and sets the Spring Security context
5. `@PreAuthorize` annotations enforce role-based access per endpoint
6. For HR-scoped data, the backend reads the HR's email from the JWT, looks up their `departmentId` , and filters results accordingly

Prerequisites

Tool	Minimum Version	Check Command
Java (JDK)	21	<code>java -version</code>
Maven	3.9+	<code>mvn -v</code> or use <code>./mvnw</code>
Node.js	18+	<code>node -v</code>
Angular CLI	18+	<code>ng version</code>
MongoDB	6+	<code>mongod --version</code>

Setup & Running Locally

Step 1 — Clone the repository

```
git clone https://github.com/hassanraheem/Stella_technology_HR_App.git
cd Stella_technology_HR_App
```

Step 2 — Start MongoDB

```
# macOS (Homebrew)
brew services start mongodb-community

# Linux
sudo systemctl start mongod

# Verify
mongosh --eval "db.runCommand({ ping: 1 })"
# Expected output: { ok: 1 }
```

Step 3 — Configure environment

```
cd demo
cp ../.env.example .env
```

Edit `demo/.env` with your values:

```
MONGO_URI=mongodb://localhost:27017/hr_management_db
JWT_SECRET=your_long_random_secret_at_least_32_hex_characters
JWT_EXPIRATION=86400000
```

Generate a secure secret: `openssl rand -hex 32`

Step 4 — Run the backend

```
# macOS / Linux (from demo/ directory)
set -a && source .env && set +a
./mvnw spring-boot:run

# Windows (PowerShell)
$env:JWT_SECRET="your_secret"; $env:MONGO_URI="mongodb://localhost:27017/hr_management_db"
.\mvnw.cmd spring-boot:run
```

The backend starts on <http://localhost:8080>

On first startup, the system seeds the admin account and the Unassigned system department automatically.

Step 5 — Run the frontend

```
cd hr-management-system-frontend
npm install      # first time only
ng serve
```

The frontend starts on **http://localhost:4200**

Step 6 — Log in

Field	Value
URL	http://localhost:4200
Email	admin@hrms.com
Password	Admin@1234

Recommended first-run order

1. Log in as **Admin**
2. Go to **Employees** → register one or more HR users (role: HR — no department needed yet)
3. Go to **Departments** → create a department and assign an HR user
4. Go to **Employees** → register regular employees, assign them to the department
5. Log out → log in as the **HR** user to manage leaves and payroll
6. Log out → log in as an **Employee** to apply for leave and view payroll

Environment Variables

Variable	Description	Default
MONGO_URI	MongoDB connection string	mongodb://localhost:27017/hr_management_db
JWT_SECRET	Secret key for signing/verifying JWT tokens	Required — no default
JWT_EXPIRATION	Token lifetime in milliseconds	86400000 (24 hours)

Security: Never commit `.env` to GitHub. The file is in `.gitignore` and will not be pushed. The `application.yml` references secrets as `${JWT_SECRET}` — not hardcoded values.

Roles & Permissions

Feature	Admin	HR	Employee
Register / Edit / Delete employee	✓	✗	✗
Create / Edit / Delete department	✓	✗	✗
Assign HR and manager to department	✓	✗	✗
View all employees (org-wide)	✓	✓	✗
View own profile	✓	✓	✓
Apply for leave	✗	✓	✓
View all leaves (org-wide)	✓ (read-only)	✗	✗
View department leaves	✗	✓	✗
Approve / reject leave	✗	✓ (dept only)	✗
Generate payroll	✗	✓ (dept only)	✗
Mark payroll paid / unpaid	✗	✓ (dept only)	✗
View all payroll (org-wide)	✓ (read-only)	✗	✗
View own payroll	✓	✓	✓
Add performance note	✓	✗	✗
View employee history	✓	✓	✓ (own only)